

From Raw Data to Reliable Models

Data Preparation, Validation & Performance
Evaluation

COMP6246: Machine Learning Technologies (MSc)

Dr Indu Bodala · i.p.bodala@soton.ac.uk

29 September 2026

The workflow is one connected system

- By the end of this lecture, you will be able to
 - Encode, scale and engineer features for model-ready input
 - Split data before preprocessing and protect the final test set
 - Build leakage-safe pipelines and match validation to deployment
 - Interpret metrics and set thresholds to reflect error costs

Encoding Categorical Variables

- Why do we need encoding?
 - Most machine learning algorithms work with numerical values, not text labels.
 - E.g. Algorithms cannot directly perform mathematical operations on categories such as *Premium*, *Married*, or *PhD*.
 - Goal: Convert categorical data into numerical representations while preserving the information.

Subscription Type	Marital Status	Education
Premium	Single	High School
Basic	Married	Masters
Premium	Divorced	PhD

Common Encoding Methods I

- Binary Encoding
 - Use when there are only two categories
 - E.g. Subscription Type

Subscription Type	Encoded
Basic	0
Premium	1

- Provides a simple and compact representation.
- Note: The choice of 0 and 1 does not imply that Premium is "better" than Basic.

Common Encoding Methods II

- One-Hot Encoding
 - Use when categories have no natural order
 - E.g. Marital Status

Marital Status	Single	Married	Divorced
Single	1	0	0
Married	0	1	0
Divorced	0	0	1

- The algorithm treats all categories as equally distinct.
- Encoding as Single=1, Married=2, Divorced=3 would have incorrectly suggest an ordering.

Common Encoding Methods III

- Ordinal Encoding
 - Use when categories have a meaningful order
 - E.g. Education

Education	Encoded
High School	1
Bachelor's	2
Master's	3
PhD	4

- Preserves the natural progression of educational qualifications.
- Note: Only use ordinal encoding when the ordering is meaningful and relevant.

Feature Scaling

- Problem: Unequal feature ranges

Feature	Typical Range
Age	14 – 60
Income	20,000 – 120,000

- Some machine learning algorithms may give **more importance to Income simply because of its larger magnitude**, not because it is more informative.
 - Algorithms that **measure distances** are sensitive to feature scales; algorithms that make rule-based splits generally are not.
- Solution: Scale the features
 - Transform features so they have comparable ranges.

Feature Scaling Techniques

- Min-max scaling

$$X_{\text{scaled}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

- Rescales values to $[0,1]$; easy to interpret
 - Preserves relative distances/original distribution shapes
 - Use when there are no extreme outliers
 - Often used in neural networks
- Standardization (z-score scaling)

$$X_{\text{scaled}} = \frac{X - \mu}{\sigma}$$

- Rescales values to mean = 0 and stdev = 1
 - More robust when values span large ranges
 - Less affected by outliers
 - Used in SVMs, Logistics Regression, PCA

Feature Engineering

- Feature engineering transforms raw variables into representations that expose useful patterns to a model.

Raw input	Engineered feature	Potential signal
Timestamp	Days since last purchase	Recency
Area + rooms	Area per room	Space density
Clicks + views	Click-through rate	Relative engagement

DERIVED-FEATURE EXAMPLE

$$\text{Area per room} = \text{Area} \div \text{Number of rooms}$$

The ratio compares usable space across differently sized properties.

- A useful feature should be predictive, available at inference time, stable, and explainable.

Common Feature Engineering Techniques

- Numerical transformations
 - Ratios, differences, rates, logarithms and bins
 - Interaction or polynomial terms
- Time and sequence features
 - Calendar, elapsed-time, lag and rolling features
- Categorical and text features
 - Rare-category grouping, counts and domain aggregates

Formula examples

RATIO

$$r = x_1 \div x_2$$

INTERACTION

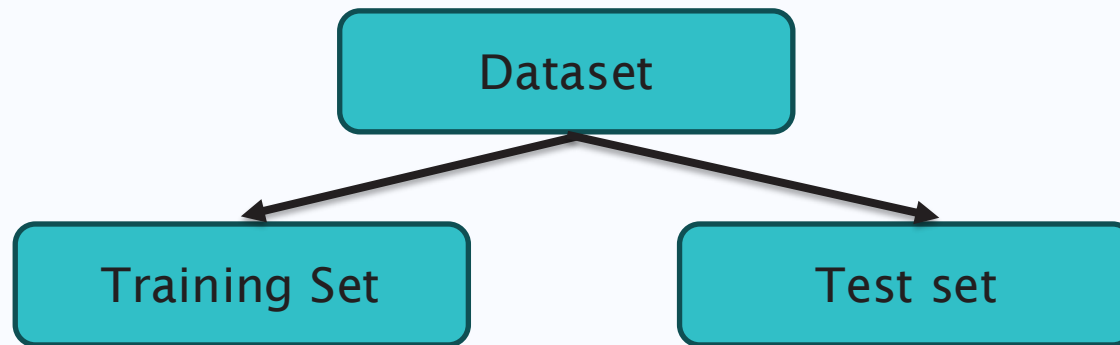
$$z = x_1 \times x_2$$

ROLLING MEAN

$$\bar{x}_t = (x_t + \dots + x_{t-k+1}) \div k$$

Split before you inspect or transform

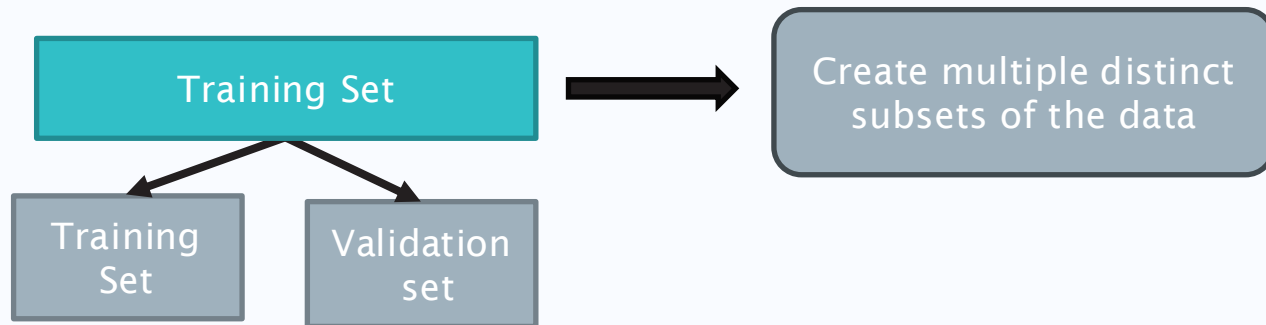
- Protect the evaluation before learning anything from the data.
 - Create the holdout split before imputation, scaling, encoding or feature selection.



- Use the Training set for training and validation decisions.
- Touch the final test set once, after the pipeline and settings are fixed.

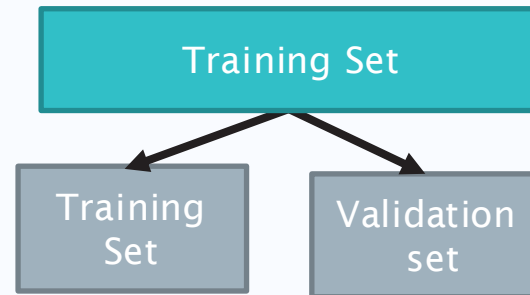
Cross Validation

- N-fold cross-validation
 - You start by creating multiple distinct subsets of data



- At each cross-validation fold, train on random $n - 1$ subsets and test on the remaining set.
- The result is an array containing n evaluation scores

Building leakage-safe pipelines



- Each validation fold must behave like genuinely unseen data.
- If preprocessing is fitted once on the full dataset, the validation rows influence those fitted values before they are scored.
- The model has indirectly “seen” the validation fold, so the score can look better than future performance really will be. This is **data leakage**.
- Solution - place preprocessing and the estimation in one pipeline, then cross-validate that complete pipeline.

Cross Validation Strategies

- Validation must match deployment
 - The split strategy encodes what “unseen” will mean in production.
- For independent and identically distributed observations, K-fold cross-validation is a reasonable default.
- With imbalanced labels, *stratification* helps maintain class proportions.
 - E.g. for an unbalanced dataset of two classes with 80/20 label split, aim for a similar label split across the folds
- When multiple rows belong to the same person, device or organisation, keep each group in only one fold, i.e. *Grouped folds*
 - prevents data leakage and provides a more realistic estimate of how well the model generalizes to new entities.

Forward splitting for temporal data

- For temporal (time-series) data, use Forward splits to respect the direction of time
 - Train on earlier observations, validate on later observations.
 - This strategy avoids *look-ahead bias* (data leakage) and overly optimistic performance estimates.
 - Expanding window – training set grows over time
 - Rolling window – training window stays a fixed size and moves forward through time.

EXPANDING WINDOW

	t1	t2	t3	t4	t5	t6	t7	t8
F1	TRAIN	TRAIN	VALIDATE	VALIDATE	OUTSIDE	OUTSIDE	OUTSIDE	OUTSIDE
F2	TRAIN	TRAIN	TRAIN	TRAIN	VALIDATE	VALIDATE	OUTSIDE	OUTSIDE
F3	TRAIN	TRAIN	TRAIN	TRAIN	TRAIN	TRAIN	VALIDATE	VALIDATE

ROLLING WINDOW

	t1	t2	t3	t4	t5	t6	t7	t8
F1	TRAIN	TRAIN	VALIDATE	VALIDATE	OUTSIDE	OUTSIDE	OUTSIDE	OUTSIDE
F2	OUTSIDE	OUTSIDE	TRAIN	TRAIN	VALIDATE	VALIDATE	OUTSIDE	OUTSIDE
F3	OUTSIDE	OUTSIDE	OUTSIDE	OUTSIDE	TRAIN	TRAIN	VALIDATE	VALIDATE

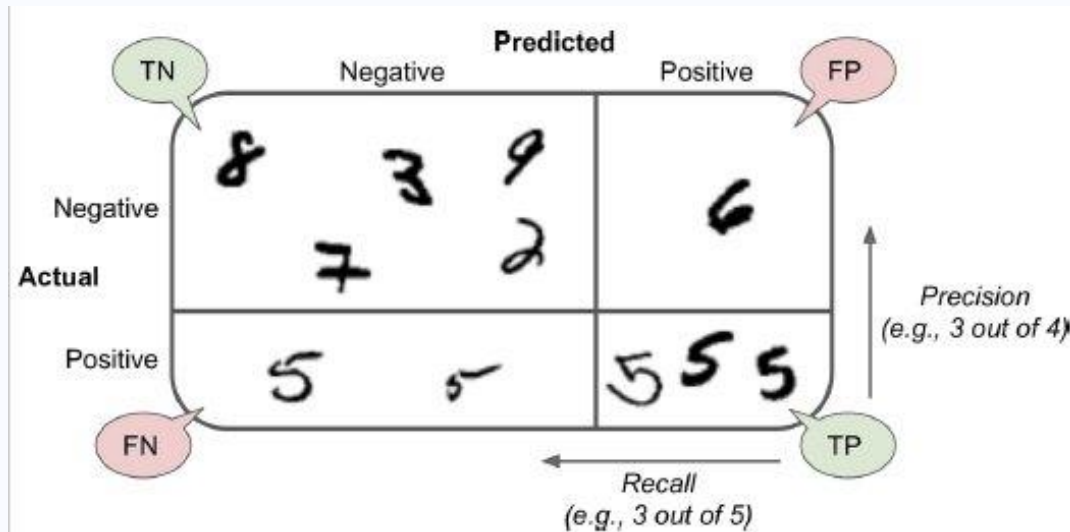
 TRAIN
  VALIDATE
  OUTSIDE

Accuracy as a performance measure (recap)

- Example: asthma attack prediction from cough sound
 - 99% of cases: no attack after coughing
 - 1%: asthma attack
- We train a model and get 99% accuracy
 - Is this good enough?
- Think about this: a very dumb algorithm – always says “no”
 - And still is accurate 99% of the time
- Reason?
 - An imbalanced (skewed) dataset
- Solution: A new metric which can capture misclassification separately for each class instead of showing the aggregated accuracy.

Confusion Matrix (recap)

- Example: classify whether a hand-written digit is 5 (i.e., a binary classifier)
 - True positive (TP): “Yes” data correctly predicted as “Yes”
 - True negative (TN): “No” data correctly predicted as “No”
 - False positive (FP): “No” data incorrectly predicted as “Yes” (Type I error)
 - False negative (FN): “Yes” data incorrectly predicted as “No” (Type II error)



A confusion matrix diagram for a binary classifier predicting whether a hand-written digit is 5. The matrix is divided into four quadrants by a horizontal line (Actual) and a vertical line (Predicted). The top-left quadrant (Actual Negative, Predicted Negative) contains three handwritten digits: 8, 3, and 9. The top-right quadrant (Actual Negative, Predicted Positive) contains one handwritten digit: 6. The bottom-left quadrant (Actual Positive, Predicted Negative) contains two handwritten digits: 5 and 5. The bottom-right quadrant (Actual Positive, Predicted Positive) contains three handwritten digits: 5, 5, and 5. Callouts with arrows point to each quadrant: TN (True Negative) for the top-left, FP (False Positive) for the top-right, FN (False Negative) for the bottom-left, and TP (True Positive) for the bottom-right. A horizontal arrow labeled 'Recall (e.g., 3 out of 5)' points from the right towards the bottom-left quadrant. A vertical arrow labeled 'Precision (e.g., 3 out of 4)' points upwards from the bottom-right quadrant.

	Predicted	
	Negative	Positive
Negative	8, 3, 9	6
Positive	5, 5	5, 5, 5

Precision, Recall and F1-score (recap)

- $Precision = \frac{TP}{TP+FP}$; ratio of correct “Yes” predictions
- $Recall = \frac{TP}{TP+FN}$; ratio of “Yes” correctly predicted
- $F1 = \frac{2}{\frac{1}{precision} + \frac{1}{recall}} = 2 \times \frac{precision \times recall}{precision + recall} = \frac{TP}{TP + \frac{FN+FP}{2}}$
 - F1 captures both precision and recall; harmonic mean of the two values
 - High only if both values are similarly high

F1 variants weight errors differently

Macro = classes · Weighted = prevalence · Micro = predictions

MACRO F1

Equal voice per class

CALCULATION

Mean of per-class F1

BEST WHEN

Minority-class performance must stay visible.

WATCH-OUT

Rare classes can move the score sharply.

WEIGHTED F1

Voice follows class size

CALCULATION

Support-weighted mean of class F1

BEST WHEN

The summary should match the observed class mix.

WATCH-OUT

Common classes can hide weak rare classes.

MICRO F1

Equal voice per prediction

CALCULATION

Pool all TP, FP and FN, then compute one F1

BEST WHEN

You need one overall classification score.

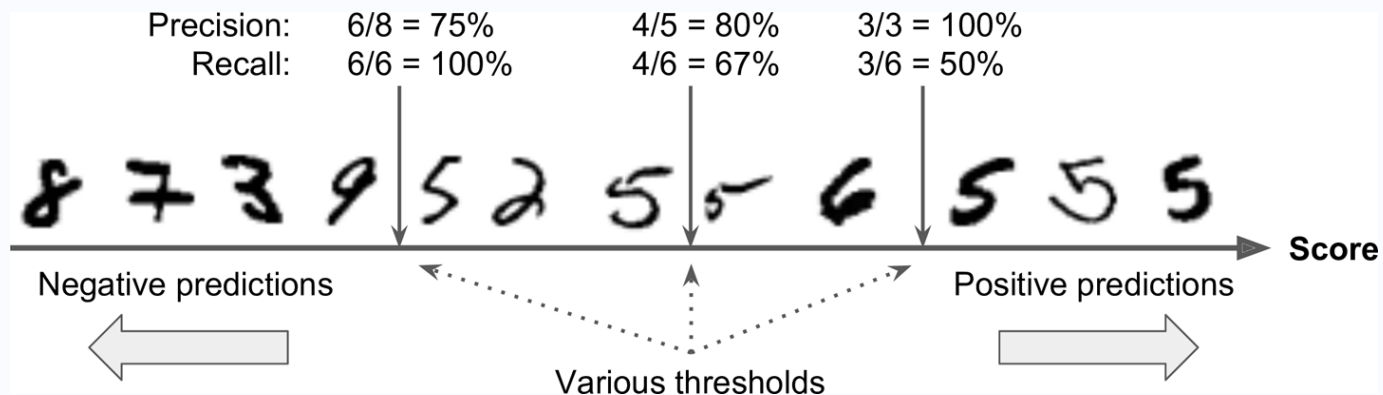
WATCH-OUT

The largest classes dominate the result.

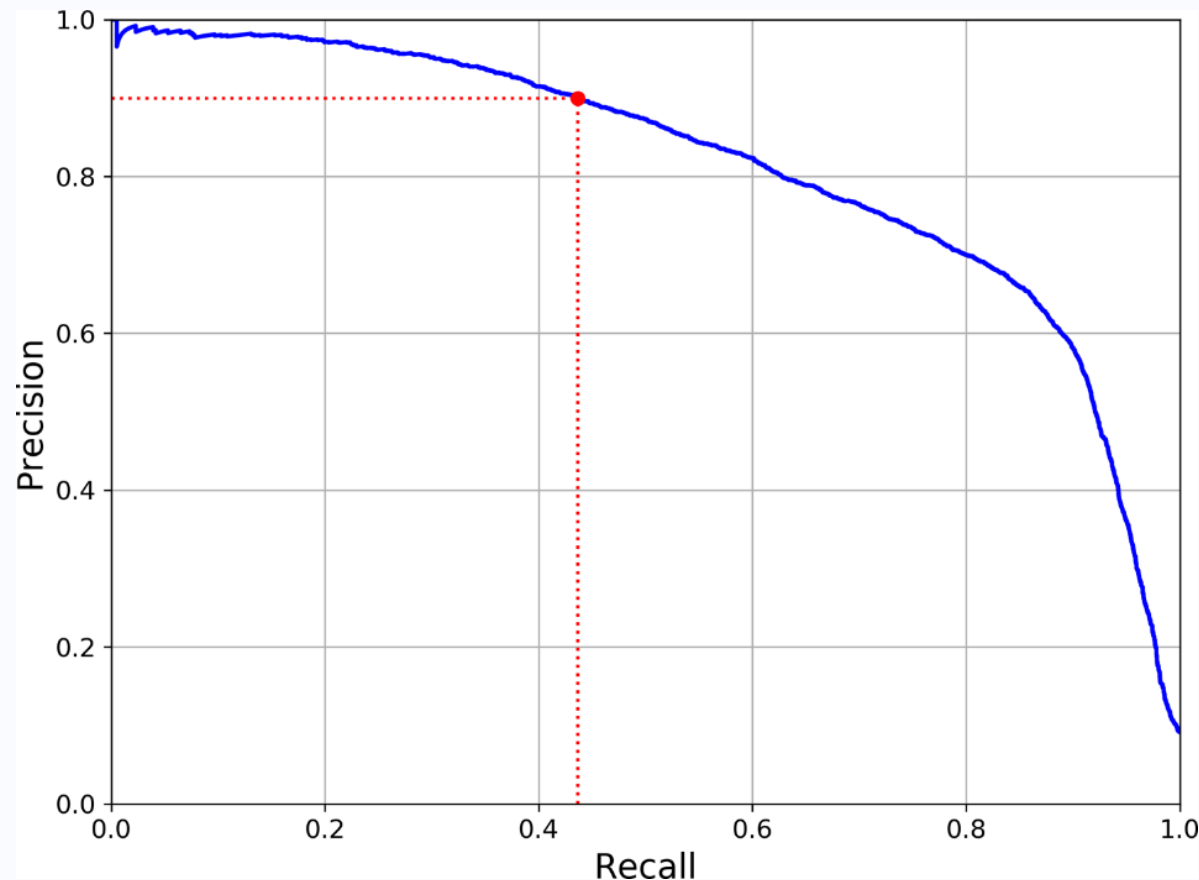
DIAGNOSTIC Macro well below weighted or micro usually signals weak minority-class performance.

Precision/Recall tradeoff

- We may sometimes prefer Precision over recall or vice-versa
 - Example: child-friendly video detection: prefer to reject many good videos (low recall) but keeps only safe ones (high precision)
- Moving the decision threshold can change the precision and recall values
 - If the threshold increases, precision increases and recall decreases
 - If threshold decreases, recall increases and precision decreases



Choose the threshold on the precision-recall curve



Each point is one threshold. Up = more precision. Right = more recall.

SHIFT THE THRESHOLD

Lower threshold = more recall, usually less precision.

SELECT BY COST

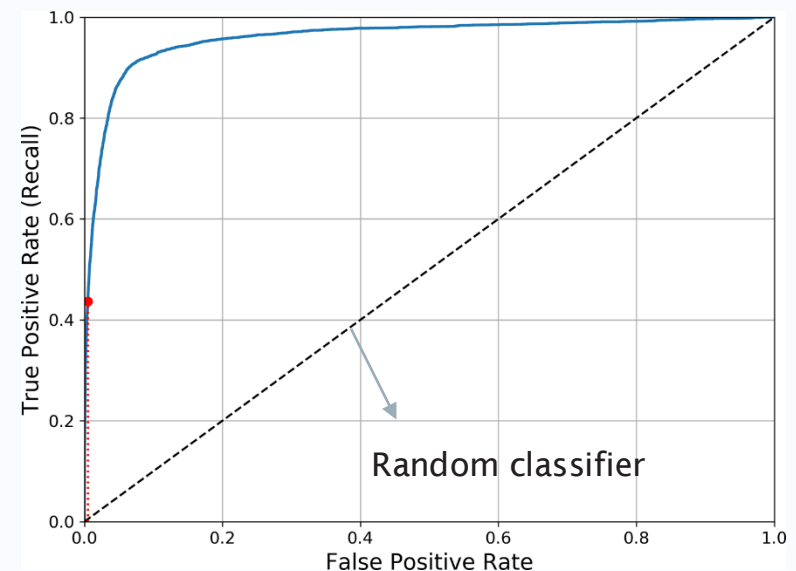
False positives costly? favour precision.

False negatives costly? favour recall.

ILLUSTRATIVE POINT precision ≈ 0.90 at recall ≈ 0.44 .

ROC curve

- Plots the *true positive rate* (also known as *recall* or *sensitivity*) against the *false positive rate* or $1 - \text{true negative rate}$ or $1 - \text{specificity}$
- ROC curve plots *sensitivity* vs $1 - \text{specificity}$
- You can compare the classifiers by measuring the *Area under the curve (AUC)*
- Preferred when looking at balanced datasets or when you want to prioritize both positive and negative classes



Summary

Reliable models require a disciplined workflow

- | | | |
|----|----------------------------------|---|
| 01 | PREPARE MODEL-READY INPUT | Encode categories by meaning, scale when the algorithm is sensitive, and engineer features that will exist at inference time. |
| 02 | PROTECT THE EVALUATION | Split first. Fit every transform on training data only, and keep preprocessing with the estimator in one pipeline. |
| 03 | VALIDATE LIKE DEPLOYMENT | Choose K-fold, stratified, grouped or forward splits to match how genuinely unseen data will arrive. |
| 04 | MEASURE WHAT MATTERS | Use error-aware metrics, compare F1 variants and set thresholds by cost—then touch the final test set once. |

Next Lecture: Decision Trees (MS)

Questions?